

SPARSE CHOLESKY ON GRAPH LAPLACIANS

VIDUSHI MITTAL

UNIVERSITY OF CALIFORNIA, DAVIS

FINAL PROJECT, MATH 226B: NUMERICAL METHODS

October 6, 2026

1 Introduction to Graphs

A weighted graph is defined as follows: $G = (V, E, w)$, where V is a set of vertices, and E is a set of edges. Edges are defined on $V \times V$, where $e \in E$ is represented by a pair $e = (v_1, v_2) \in V \times V$, indicating that an edge exists between a pair of vertices (v_1, v_2) . A weighted graph allows each edge to hold a numerical value, which we refer to as a weight. The assignment of weights can be thought of as a function $w : V \times V \rightarrow \mathbb{R}$ which assigns a positive real value $w(e)$ to each $e \in E$. In this project we examine multi-graphs, which allow for multiple edges between the same pair of vertices. The number of edges connected to a vertex $v \in V$ is called the degree of the vertex, $d(v)$, i.e. the degree of a vertex indicates the number of vertices that v shares an edge with.

The graphs we consider will be undirected. Undirected means that any edge is symmetric. If the vertex v_1 is connected to vertex v_2 by some edge, that same edge connects v_1 to v_2 . A subgraph S of the graph G , $S \subseteq G$, is a graph $S = (U, W)$ where $U \subseteq V$, $W \subseteq E$. A complete graph is a graph in which every all pairs of vertices in V two vertices in the vertex set are connected by an edge. A clique of a graph G is a complete subgraph.

2 Graph Laplacians

We can frame graphs as matrices, by way of adjacency, degree, and Laplacian matrices. The adjacency matrix indicates to us whether or not two vertices are connected by an edge. Explicitly, given a graph $G = (V, E)$, the adjacency matrix A has dimension $|V| \times |V|$. Each row and column of A corresponds to a vertex $v \in V$ i.e. the i^{th} row will correspond to the vertex v_i and the j^{th} column will correspond to the vertex v_j . To define the entries, $A_{i,j} = \sum w(v_i, v_j)$ where the row vertex v_i shares an edge (possibly multiple edges) with the column vertex v_j . Note that $w(v_i, v_j)$ will always be 1 or 0 if the graph is unweighted. Since our graph is undirected, for $v_i, v_j \in V$, v_i and v_j only share an edge if v_j and v_i share an edge, so A must be symmetric. The degree matrix, D , is a diagonal matrix of dimension $|V| \times |V|$. Each diagonal entry $D_{i,i}$ corresponds to exactly one vertex $v_i \in V$, where $D_{i,i} = d(v_i)$.

The graph Laplacian is then defined by $D - A$. We may also consider an equivalent definition of the Laplacian: $L = BWB^T$ where W is the weight matrix of our graph's collection of multi-edges and B is known as the incidence matrix. W accounts for the *cumulative* edge weight between any pair of vertices, which is the same as our (weighted) adjacency matrix A . The incidence matrix has dimension $|V| \times |E|$, where each row corresponds to a vertex and each column corresponds to an edge. An entry is 1 if the row's corresponding vertex is an endpoint of the edge corresponding to the column. Otherwise, the entries are 0. Since any entry the Laplacian really only depends on the sum of weights of multiedges between the associated pair of vertices, we will express the Laplacian as such: For a pair of vertices $v_1, v_2 \in V$ that share any edge e , we define $b_{v_1, v_2} = b_e = \lambda_{v_2} - \lambda_{v_1}$, where λ_i is the i^{th} standard basis vector of $\mathbb{R}^n$. In doing this, we are assigning an arbitrary sign to each multi-edge of G , which would be reflected in the incidence matrix B . This allows us to define the Laplacian of a multi-edge e as $w(e)b_e b_e^T$, and express the graph Laplacian as

$$L = \sum_{e \in E} w(e) b_e b_e^T.$$

By construction, the graph Laplacian is a symmetric and diagonally dominant matrix. The symmetry follows naturally from the symmetry of both A and D . To show diagonal dominance of L , consider some $v_i \in V$. Then, since the i^{th} row of A has an entry of 1 in each of the columns which correspond to a vertex that v_i is connected to, we expect the sum of the i^{th} row of A to be $d(v_i)$. Thus, we expect $D_{i,i} = \sum_{j=1}^{|V|} A_{i,j}$, which tells us $L_{i,i} \geq \sum_{j \neq i} |L_{i,j}|$, giving us diagonal dominance of L .

3 Motivation

The Laplacian comes with some useful applications in various graphs. Since L is symmetric and diagonally dominant, L is positive semidefinite, and must have positive eigenvalues. We note that, since the rows of the Laplacian sum to 0, the algebraic multiplicity of the eigenvalue 0 indicates the number of connected components of the graph. If the second smallest eigenvalue is positive, then this indicates the graph is connected. By utilizing clever choices for b , solving the linear system $Lx = b$ can also yield some interesting results. For example, we can set b to be a zero vector, with 1 in entry v_i , and solving $Lx = b$ yields the similarity of all other nodes to node i . Ability to solve linear systems with the Laplacian may also assist in finding eigenvalues and eigenvectors. By finding a faster way to complete the linear system step of inverse iteration, we gain efficiency in solving for eigenvectors. Eigenvectors associated to the smallest eigenvalues can be quite useful. For a key example, the smallest k eigenvectors can be used as a column for a matrix, and we can apply k -means to that matrix to cluster the graph into k clusters.

For the purpose of this project, we consider an undirected weighted graph and its Laplacian, and, rather than relying on more costly methods like Cholesky, we turn to a new algorithm which reduces computational cost by exploiting the sparse structure of the graph Laplacian and allows us to solve linear systems in nearly linear time.

4 Standard Cholesky factorization on Laplacians

On graph Laplacians, we can intuitively understand Cholesky factorization as removing vertices while still keeping the structure of the graph. For each vertex removed, we update the entries of the neighbors to maintain the idea of their connection to the removed vertex. This, however, creates submatrices that have only nonzero entries with each iteration, or, cliques, meaning the matrix remains quite dense and computationally heavy. This occurs even when beginning with a sparse matrix, since zero entries quickly become nonzero in this process. This is called *fill-in*. Even when beginning with a sparse Laplacian, this process can run up to $O(n^3)$. To solve this, we aim to compute an approximate Cholesky factorization without creating such a dense matrix, while preserving necessary information with each step. We ask ourselves if we would consider losing accuracy to gain computational efficiency.

5 Introducing *SPARSECHOLESKY* and *CLIQUESAMPLE*

*[2] presents an alternate way to approach the problem of solving linear systems with the Laplacian. By compromising some accuracy, the efficiency of computation is brought to near-linear time. Rather than each iteration adding a clique in the graph, for an iteration removing vertex v , we add only $d(v)$ edges, by way of random choice.

The paper presents two algorithms, *CLIQUESAMPLE* and *SPARSECHOLESKY*. *CLIQUESAMPLE*, presents an alternative method to adding an entire clique with each iteration. Rather than introducing an edge between every pair of neighbors of the vertex being deleted, it randomly samples the clique, considering edge weights, and chooses only $d(v)$ edge additions with the removal of each $v \in V$. This algorithm is called by *SPARSECHOLESKY* as it works through Cholesky factorization on the Laplacian. Rather than achieving an exact Cholesky factorization, this method does lose some accuracy.

SPARSECHOLESKY takes as input the Laplacian, expressed as a sum, as well as scalars $\delta > 1$ and $0 < \epsilon \leq \frac{1}{2}$. It

returns matrices $P, \mathcal{L}, D$ which yield an approximate Cholesky decomposition $P\mathcal{L}D\mathcal{L}^T P^T$. Though we do not deeply discuss the loss in accuracy here, the method results in an accuracy

$$(1 - \varepsilon)L \leq P\mathcal{L}D\mathcal{L}^T P^T \leq (1 + \varepsilon)L$$

with probability at least $1 - \frac{2}{n^\delta}$. Increasing the δ increases the probability of accuracy within the bound above, while decreasing ε increases the accuracy, tightening the bound above.

The algorithm works by first choosing a random permutation π as the order in which to delete vertices. Eliminating vertices one at a time, the algorithm deletes all edges incident to the current vertex, and replacing those edges by the edges given as the result of the use of *CLIQUESAMPLE*.

CLIQUESAMPLE takes as input the Laplacian, and a single vertex v . The goal is to output $d(v)$ edges by way of their Laplacians. In each iteration i from 1 to $d(v)$, we first use the edge weights of the edges incident to v as the probability of selecting that edge to select an edge e_1 . This is done so that edges with higher weight are more likely to be chosen. We next select a random edge incident to v to be e_2 . We confirm that the other endpoints of e_1 and e_2 , which we call u_1, u_2 respectively, are not equal, and assign

$$Y_i = \frac{w(e_1)w(e_2)}{w(e_1) + w(e_2)} b_{u_1, u_2} b_{u_1, u_2}^T.$$

Effectively, this creates an $n \times n$ matrix of zeros, with $\frac{w(e_1)w(e_2)}{w(e_1) + w(e_2)}$ in the (u_1, u_1) and (u_2, u_2) positions, and $-\frac{w(e_1)w(e_2)}{w(e_1) + w(e_2)}$ in the (u_1, u_2) and (u_2, u_1) positions. However, rather than storing each entry of Y_i in a matrix, we simply store the Laplacian in this manner. Though we update four entries, this process represents the addition of one edge between u_1 and u_2 , and updating the diagonal entries to reflect this and keep the Laplacian diagonally dominant. In the case that $u_1 = u_2$, we simply let $Y_i = 0$. This process expresses the edge we are adding as its Laplacian, and *CLIQUESAMPLE* ultimately returns

$$\sum_i Y_i,$$

the sum of the Laplacians of the new edges.

With each iteration, the Laplacian becomes the sum of the Laplacian in the previous iteration added to the result given by *CLIQUESAMPLE*. We also subtract the entries corresponding to the vertex of the current iteration, $\pi(k)$ to account for the vertex deletion. Our resulting Laplacian at the end of this process holds the sums of all the edges added along the way.

Let us consider the construction of the matrices $P, \mathcal{L}$, and D . P is simply the permutation matrix corresponding to the permutation π we use to order the deletion of our vertices. The diagonal matrix entry corresponding to the vertex being considered at the current iteration is assigned the value of that vertex in the Laplacian of the previous iteration. In other words, we assign at k^{th} iteration:

$$D(\pi(k), \pi(k)) = S^{(k-1)}(\pi(k), \pi(k)).$$

$\mathcal{L}$ is constructed by storing the columns of the eliminated vertices divided by their pivots. Meaning, the $\mathcal{L}_k$ columns is created by the $\pi(k)^{th}$ column of $S^{(k-1)}$ divided by the $D(\pi(k), \pi(k))$ we just defined above.

6 Solving $Lx = b$

Once we have applied *SPARSECHOLESKY* to obtain an approximate Cholesky factorization, say we have some vector b and we desire to solve $Lx = b$ using $P, D, \mathcal{L}$. We use iterative refinement to solve the system. We begin with an initial $x(0) = 0$. We calculate the initial residual r . We then want to solve for $(P^T D^{-1} P^T)y = r$. To do

this, we first compute Pr . We forward solve to get $s_1 = \mathbb{V}^{-1}Pr$. We apply D^{-1} to this to get $s_2 = D^{-1}s_1$. Then calculate $(\mathbb{V}^T)^{-1}s_2$. Finally reverse the permutation and get $y = P^T s_3$. We update $x^{k+1} = x^k - \frac{1}{2}y$, and continue for the desired number of iterations.

7 Code

```
import numpy as np

def gen_sparse_laplacian(n, edge_prob=0.2):
    A=np.zeros((n, n))
    for i in range(n):
        for j in range(i+1,n):
            if np.random.rand()< edge_prob:
                A[i,j]=np.random.randint(1, 11)
                A[j,i]=A[i,j]
            else:
                A[i,j]=0
                A[j,i]=0
    D=np.diag(np.sum(A, axis=1))
    L=D-A
    return L

import numpy as np

def clique_sample(S,v,n):
    #First finding all the edges that connect to the vertex v
    all_edges=S.get(v,[])
    #Making set of only edges with positive weights(actually connected to v)
    incident_edges=[(u,w) for u,w in all_edges if w>0]

    #Degree of v should be the number of incident edges with positive weight
    dv=len(incident_edges)
    #Double checking the selected vertex has edges
    if dv==0:
        return []
    else:
        #Summing total weight of edges incident to v, assigning probability
        wSv=sum(e[1] for e in incident_edges)
        prb=[e[1]/wSv for e in incident_edges]

        Y=[]
        for _ in range(dv):
            #Sampling e1 with probability proportional to weight
            e1=np.random.choice(dv, p=prb)
            u1,w1=incident_edges[e1]

            #Sampling e2 uniformly
            e2=np.random.choice(dv)
            u2,w2=incident_edges[e2]

            #Confirming the endpoints are different, and creating a new edge
            if u1!=u2:
                new_w = (w1 * w2) / (w1 + w2)
                Y.append((u1, u2, new_w))
            else:
                Y.append((u1, u2, 0))
```

```

return Y

def sparse_cholesky(L_matrix, eps, delta):
    #Setting n to be the number of vertices, and thus number of iterations
    n=L_matrix.shape[0]
    #Splitting edges into rho copies using rho = ceil(12(1+delta)^2epsilon^-2(ln(n))^2)
    rho = int(np.ceil(12 * (1+delta)**2 * (eps**-2) * (np.log(n)**2)))

    #Representing the multi-graph as adjacency list of edges
    #Want S[v]={ (neighbor of v, weight)}
    S = {i:[] for i in range(n)}
    for i in range(n):
        for j in range(i+1, n):
            weight=-L_matrix[i, j]
            if weight>0:
                for _ in range(rho):
                    w_split = weight/rho
                    S[i].append((j, w_split))
                    S[j].append((i, w_split))

    D=np.zeros(n)
    C=np.zeros((n, n))
    pi=np.random.permutation(n)

    #Want to track which vertices have been eliminated
    eliminated=set()
    #Also want to keep track of original vertex index vs permuted vertex index
    inv_pi={v:k for k,v in enumerate(pi)}

    for k in range(n-1):
        v=pi[k]
        #Getting diagonal entry, which should be the sum of entries
        #in v row, which are just edges
        D[v]=sum(e[1] for e in S[v])

        #Extracting column and normalizing
        if D[v]>0:
            #Set up column relative to vertex v
            col=np.zeros(n)
            for neighbor,weight in S[v]:
                #Considering only the neighbors that haven't been used already
                if neighbor not in eliminated:
                    col[neighbor]-=weight
            col[v]=D[v]
            C[:, k]=col/D[v]

        #Remember v got eliminated alrdsy
        eliminated.add(v)
        #Using cliquesample
        new_edges=clique_sample(S, v, n)

        #Removing edges incident to v
        neighbors_of_v=[neighbor for neighbor,weight in S[v]]
        for neighbor in neighbors_of_v:
            S[neighbor]=[(u,w) for u,w in S[neighbor] if u!=v]

```

```

    #Clearing v's adjacency list(since we eliminated vertex v)
    S[v]=[]

    #Adding clique edges to entries (u1,u1),(u1,u2),(u2,u1),(u2,u2), expressed
    #in tuple associated to u1,u2
    for u1,u2,w in new_edges:
        # Add edge (u1, u2) to the multi-graph
        S[u1].append((u2,w))
        S[u2].append((u1,w))

#Setting the final vertex
v_n=pi[n-1]
D[v_n]=sum(e[1] for e in S[v_n])

#Extracting final column from current S
if D[v_n]>0:
    col=np.zeros(n)
    for neighbor, weight in S[v_n]:
        col[neighbor]-=weight
    col[v_n]=D[v_n]
    C[:,n-1]=col/D[v_n]
else:
    C[:,n-1]=np.eye(n)[: ,v_n]

#Constructing permutation matrix P and lower triangular L
P_pi=np.eye(n)[pi]
L_out=C[:, :n]

return P_pi, L_out, D

def forward(L,b):
    n=len(b)
    y=np.zeros(n)
    for i in range(n):
        sumv=0
        for j in range(i):
            if L[i,j]!=0:
                sumv+=L[i,j]*y[j]
        d=L[i,i]
        y[i]=(b[i]-sumv)/d if d!=0 else 0
    return y

def backward(LT,b):
    n=len(b)
    x=np.zeros(n)
    for i in range(n-1,-1,-1):
        sumv=0
        for j in range(i+1,n):
            if LT[i,j]!=0:
                sumv+= LT[i,j]*x[j]
        d=LT[i,i]
        if d!=0:
            x[i]=(b[i]-sumv)/d
        else:
            x[i]= 0

```

```

    return x

def iter_solv(L_orig, P, L_chol, D, b, it=5):
    n=len(b)
    D_inv=np.zeros(n)
    x=np.zeros(n)
    for d in range(n):
        if D[d]!=0:
            D_inv[d]=(1/D[d])
        else:
            D_inv[d]=0
    for _ in range(it):
        r=L_orig @ x-b
        r_p=P.dot(r)

        #Solving system with Cholesky factorization
        s1 = forward(L_chol, r_p)
        s2 = D_inv * s1
        s3 = backward(L_chol.T, s2)
        y = P.T.dot(s3)

        #Definign the next x
        x = x-0.5*y
    return x

```

8 Results

As a test example,

```

... TEST: solvecholesky function
Original L:
[[19. -8.  0. -3. -8.]
 [-8. 18. -8.  0. -2.]
 [ 0. -8.  8.  0.  0.]
 [-3.  0.  0.  8. -5.]
 [-8. -2.  0. -5. 15.]]
Reconstructed L
[[ 0.      0.      0.      0.      0.      ]
 [ 0.      9.6933 -9.6933  0.      0.      ]
 [ 0.     -9.6933 17.6933 -0.5346 -7.4654]
 [ 0.      0.     -0.5346  6.345  -5.8104]
 [ 0.      0.     -7.4654 -5.8104 28.2759]]
Original b:
[0.3994 0.22  0.1668 0.173  0.1926]
Reconstructed b
[ 0.     -0.1199  0.2453  0.0196 -0.1991]
Relative error in solving L_approx @ x = b: 1.2302

```

Though the approximations are not exact, on a large scale, they help capture the main ideas, and may be useful in capturing the key structures of a larger network.

REFERENCES

- [1] N. J. HIGHAM, *Accuracy and Stability of Numerical Algorithms*, Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 2nd ed., 2002.
- [2] R. KYNG AND S. SACHDEVA, *Approximate gaussian elimination for laplacians-fast, sparse, and simple*, in 2016 IEEE 57th Annual Symposium on Foundations of Computer Science (FOCS), IEEE, 2016, pp. 573–582.
- [3] D. SPIELMAN, *Spectral graph theory*, in Combinatorial Scientific Computing, Chapman & Hall/CRC, 2012, ch. 16, pp. 1–30.
- [4] L. N. TREFETHEN AND D. BAU III, *Numerical Linear Algebra: Twenty-Fifth Anniversary Edition*, Society for Industrial and Applied Mathematics, Philadelphia, 2022.